



University
Mohammed VI
Polytechnic



Deliverable #: Title of Lab/Project here (e.g.: Conceptual Design)

Data Management Course
UM6P College of Computing

Professor: Karima Echihabi **Program:** Computer Engineering
Session: Fall 2025

Team Information

Team Name	
Member 1	Adam Chikri
Member 2	Amine Abdellaoui
Member 3	Douae Bouayadi
Member 4	Aya Abid
Member 5	Nissrine Benallal
Member 6	Yasmin Bouaboud
Repository Link	https://github.com/Adam-chi/Lab4-DBMS

1 Introduction

This deliverable represents Part III of the MNHS project, focusing on advanced database querying and analysis. It covers the formulation of complex business questions using Relational Algebra and their practical implementation in SQL. Additionally, it involves a thorough analysis of Functional Dependencies within the database schema, providing crucial insights into data relationships and integrity constraints essential for the Moroccan National Health Services system.

2 Requirements

The objective of this project part was to comprehensively query, analyze, and refine the MNHS relational database through multiple advanced database techniques. The requirements were multifaceted and can be detailed as follows:

Relational Algebra Formulation

- Express a predefined set of 15 non-trivial, business-oriented queries over the MNHS relational schema using the formal syntax and operators of Relational Algebra.
- Ensure that the queries address various real-world scenarios that test the database's capabilities in handling complex data relationships.
- Cover a broad range of operations including selection, projection, join operations, set operations, and aggregation scenarios.

SQL Translation and Implementation

- Translate each of the 15 Relational Algebra expressions into corresponding SQL statements.
- Ensure that the SQL queries maintain the same logical structure and output requirements as their relational algebra counterparts.
- Implement SQL queries involving multiple table joins, subqueries, aggregate functions, and complex filtering conditions.
- Incorporate advanced SQL features such as grouping, sorting, and conditional expressions to fulfill query specifications.

Functional Dependency Analysis

- Conduct a detailed analysis of the entire MNHS relational schema to identify all significant Functional Dependencies.
- Determine dependencies related to primary keys, candidate keys, and other attribute relationships within each table.
- Identify and analyze transitive and partial dependencies that may exist across the schema.

3 Methodology

This methodology describes how we designed and queried the Moroccan National Health Services (MNHS) database for Lab 4, focusing on the use of relational algebra operators and the role of functional dependencies in shaping our approach, as follows:

Selection Operator (σ)

This operator was used to filter tuples based on specific conditions. For instance, in Query 1, $\sigma_{\text{status}=\text{"Active"}}(\text{Staff})$ filters only active staff members before joining with other relations. Similarly, date-range filtering in Query 9 uses $\sigma_{01/09/2025 \leq \text{Date} \leq 30/09/2025}(\text{Clinical Activity})$ to isolate September activities.

Projection Operator (π)

This operator extracts specific attributes from relations while eliminating duplicates. It was employed in nearly every query to ensure only relevant attributes appear in the final result. For example, $\pi_{\text{Name}}(\text{Patient} \dots)$ in Query 1 returns only patient names rather than complete patient records. In Query 5, $\pi_{\text{Dep_ID}}(\sigma_{\text{HID}=1}(\text{Department}))$ projects department IDs for a specific hospital.

Join Operator ($\bowtie$)

This operator combines tuples from multiple relations based on common attributes. This was crucial for queries requiring information from multiple entities. Query 1 demonstrates chaining joins: $\text{Patient} \bowtie (\text{Staff} \bowtie \text{Clinical Activity})$ connects patients to their clinical activities through staff members. Query 9 uses $(\text{Clinical Activity} \bowtie \text{Department}) \bowtie \text{Hospital}$ to link activities to hospital locations.

Rename Operator (ρ)

This operator assigns new names to relations or attributes, enabling self-joins and disambiguation. Query 8 demonstrates self-join renaming: $\rho(R1, \text{Clinical Activity})$ and $\rho(R2, \text{Clinical Activity})$ create two distinct instances of the same relation for comparison.

Union ($\cup$)

This operator was selected for “or” conditions; it combines tuples from two relations with compatible schemas, eliminating duplicates. Query 2 demonstrates union: $\text{Active Staff} \cup \text{Prescription Staff}$ merges staff who are either active or have issued prescriptions. Query 3 uses $\text{Benguerir Hospitals} \cup \text{Cardiology Hospitals}$ to find hospitals meeting either criterion.

Intersection ($\cap$)

This operator was selected for “and” conditions; it returns tuples present in both relations. Query 4 uses intersection to find hospitals with both specialties:

$$\pi_{\text{HID}}(\sigma_{\text{Specialty}=\text{"Cardiology"}}(\text{Department})) \cap \pi_{\text{HID}}(\sigma_{\text{Specialty}=\text{"Pediatrics"}}(\text{Department})).$$

Query 9 employs intersection to identify activities meeting both temporal and geographic constraints.

Difference (−)

This operator identifies tuples in the first relation but not the second. For instance, Query 12 uses difference to find staff without scheduled appointments: $\text{Staff} - \text{Staff_At_Date}$.

Division (/)

This operator identifies tuples from one relation associated with all tuples in another relation. While not directly supported in SQL, division was implemented using projection, Cartesian product, and difference. Query 5 demonstrates division: A/B finds staff who worked in every department of a specific hospital. The formula

$$A/B = \pi_{\text{Staff_ID}}(A) - \pi_{\text{Staff_ID}}((\pi_{\text{Staff_ID}}(A) \times B) - A)$$

implements division using fundamental operators.

Aggregation Operator (γ)

This operator performs grouping and aggregation functions like COUNT, SUM, AVG, and MAX. Query 7 uses $\gamma_{\text{Staff_ID}, \text{COUNT}(*)}(\text{Clinical Activity})$ to count activities per staff member. Query 13 employs nested aggregation: $\gamma_{\text{AVG}(\text{count})}(\text{Activity Count})$ computes the global average after initial grouping. Query 14 combines grouping with MAX: $\gamma_{\text{Staff_ID}, \text{MAX}(\text{Count1})}(\text{Completed Appointment})$ identifies maximum counts per staff.

Cartesian Product ($\times$)

This operator creates all possible tuple combinations from two relations; it was necessary when join conditions weren't based on equality or when implementing division. For example, Query 7 uses $S1 \times S2$ to create all staff pairs for activity count comparison. Query 8 employs $R1 \times R2$ for self-joining Clinical Activity to find patients with multiple staff interactions.

SQL Translation Strategy

The translation of relational algebra to SQL maintains logical equivalence. Selection operators translated to WHERE clauses, projections to SELECT statements, joins to JOIN clauses, and set operations to UNION, INTERSECT, and EXCEPT keywords. Aggregation operators mapped to GROUP BY with aggregate functions.

The Functional Dependencies

FDs describe how certain attributes uniquely determine others, which is crucial for:

- Identifying primary and candidate keys (e.g., IID and CIN in Patient)
- Ensuring data integrity (e.g., no duplicate patients or staff)

-
- Guiding join conditions (e.g., using foreign keys for joins)
 - Optimizing queries by knowing which attributes are unique and which can be used for grouping and filtering

4 Implementation & Results

Relational Algebra:

1. $\pi_{Name}(Patient \bowtie (\sigma_{status="Active"}(Staff) \bowtie Clinical\ Activity))$
2. $\rho(Active_Staff, \pi_{Staff_ID}(\sigma_{Status="Active"}(Staff)))$
 $\rho(Prescription_Staff, \pi_{Staff_ID}(Clinical\ Activity \bowtie Prescription))$
 $Active_Staff \cup Prescription_Staff$
3. $\rho(Benguerir_Hospitals, \pi_{HID}(\sigma_{City="Benguerir"}(Hospital)))$
 $\rho(Cardiology_Hospitals, \pi_{HID}(\sigma_{Specialty="Cardiology"}(Department)))$
 $Benguerir_Hospitals \cup Cardiology_Hospitals$
4. $\rho(Pediatrics_Hospitals, \pi_{HID}(\sigma_{Specialty="Pediatrics"}(Department)))$
 $\rho(Cardiology_Hospitals, \pi_{HID}(\sigma_{Specialty="Cardiology"}(Department)))$
 $\pi_{HID}(\sigma_{Specialty="Cardiology"}(Department)) \cap \pi_{HID}(\sigma_{Specialty="Pediatrics"}(Department))$
5. $\rho(A, Works_in)$
 $\rho(B, \pi_{Dep_ID}(\sigma_{HID=1}(Department)))$
 $A/B = \pi_{Staff_ID}(A) - \pi_{Staff_ID}((\pi_{Staff_ID}(A) \times B) - A)$
6. $\rho(A, \pi_{Staff_ID, CAID}(Clinical\ Activity))$
 $\rho(B, \pi_{CAID}(\sigma_{Dep_ID=2}(Clinical\ Activity)))$
 $A/B = \pi_{Staff_ID}(A) - \pi_{Staff_ID}((\pi_{Staff_ID}(A) \times B) - A)$
7. $\rho(S1, \gamma_{Staff_ID, (COUNT(CAID) \rightarrow count1)}(Clinical\ Activity))$
 $\rho(S2, \gamma_{Staff_ID, (COUNT(CAID) \rightarrow count2)}(Clinical\ Activity))$
 $\pi_{sid1, sid2}(\sigma_{count1 > count2}(S1 \times S2))$
8. $\rho(R1, Clinical_Activity)$
 $\rho(R2, Clinical_Activity)$
 $\pi_{IID}(\sigma_{R1.IID=R2.IID \wedge R1.Staff_ID \neq R2.Staff_ID}(R1 \times R2))$
9. $\rho(September_Activity, \pi_{CAID}(\sigma_{01/09/2025 \leq Date \wedge Date \leq 30/09/2025}(Clinical\ Activity)))$
 $\rho(Benguerir_Activity, \pi_{CAID}(\sigma_{City="Benguerir"}((Clinical\ Activity \bowtie Department) \bowtie Hospital)))$
 $September_Activity \cap Benguerir_Activity$
10. $\rho(R1, Clinical_Activity \bowtie Prescription)$
 $\rho(R2, Clinical_Activity \bowtie Prescription)$
 $\pi_{Staff_ID}(\sigma_{R1.Staff_ID=R2.Staff_ID \wedge R1.CAID \neq R2.CAID}(R1 \times R2))$
11. $\rho(R1, Clinical_Activity \bowtie \sigma_{Status="Scheduled"}(Appointment))$
 $\rho(R2, Clinical_Activity \bowtie \sigma_{Status="Scheduled"}(Appointment))$
 $\pi_{IID}(\sigma_{R1.IID=R2.IID \wedge R1.Dep_ID \neq R2.Dep_ID}(R1 \times R2))$

12. $\rho(Staff, \pi_{Staff_ID}(Staff))$
 $\rho(Staff_At_Date, \pi_{Staff_ID}(\sigma_{Date="06/11/2025" \wedge Status="Scheduled"}(ClinicalActivity \bowtie Appointment)))$
 $Staff - Staff_At_Date$

13. $\rho(Activity_Count, \gamma_{Dep_ID, (COUNT(CAID) \rightarrow Dep_count)}(Clinical_Activity))$
 $\rho(Global_AVG, \gamma_{(AVG(Dep_count) \rightarrow Global_Average)}(Activity_Count))$
 $\pi_{Dep_ID}(\sigma_{Dep_count < Global_AVG.Global_Average}(Activity_Count))$

14. $\rho(Completed_Appointment, \gamma_{Staff_ID, IID, (COUNT(CAID) \rightarrow Completed_n)}(Clinical_Activity \bowtie$
 $\sigma_{Status="Completed"}(Appointment)))$
 $\rho(Max_Count, \gamma_{Staff_ID, (MAX(Completed_n) \rightarrow max_n)}(Completed_Appointment))$
 $\rho(Joined_tables(1 \rightarrow Staff_ID1, 4 \rightarrow Staff_ID2), Completed_Appointment \times Max_Count)$
 $\pi_{Staff_ID, IID}(\sigma_{max_n = Completed_n \wedge Staff_ID1 = Staff_ID2}(Joined_tables))$

15. $\rho(R1, \sigma_{01/01/2025 \leq Date \wedge Date \leq 31/12/2025}(Emergency))$
 $\rho(R2, \gamma_{IID, COUNT(CAID) \rightarrow Count}(Clinical_Activity \bowtie R1))$
 $\pi_{IID}(\sigma_{Count \geq 3}(R2))$

SQL Code Implementation

We have uploaded alongside the PDF, in the repository, the .SQL File that contains all the queries listed above, translated into SQL.

Functional Dependencies:

Patient Relation

- **Schema:** Patient(IID: INTEGER, CIN: STRING, Name: STRING, Sex: STRING, Birth_Date: STRING, Blood_Group: STRING, Phone: INTEGER, **PRIMARY KEY (IID)**)
- **Notation:** $\{I, Cin, N, S, B, Bl, P\}$
- **Functional Dependencies:**
 - $I \rightarrow Cin, N, S, B, Bl, P$
 - $Cin \rightarrow I, N, S, B, Bl, P$

Staff Relation

- **Schema:** Staff(Staff_ID: INTEGER, Name: STRING, Status: STRING, **PRIMARY KEY (Staff_ID)**)
- **Notation:** $\{S, N, St\}$
- **Functional Dependencies:**
 - $S \rightarrow N, St$

Contact Location Relation

- **Schema:** Contact_Location(CLID: INTEGER, City: STRING, Province: STRING, Phone: STRING, Street: STRING, Postal_Code: STRING, Number: INTEGER, **PRIMARY KEY (CLID)**)
- **Notation:** $\{C, Ci, Pr, Ph, St, Po, Nb\}$
- **Functional Dependencies:**
 - $C \rightarrow Ci, Pr, Ph, St, Po, Nb$
 - $Po \rightarrow Ci, Pr, St, Nb$
 - $Ci, Pr \rightarrow Po$

Insurance Relation

- **Schema:** Insurance(INSID: INTEGER, Type: STRING, **PRIMARY KEY (INSID)**)
- **Notation:** $\{In, T\}$
- **Functional Dependencies:**
 - $In \rightarrow T$

Expense Relation

- **Schema:** Expense(EXID: INTEGER, Total: INTEGER, CAID: INTEGER, INSID: INTEGER, **PRIMARY KEY (EXID)**, FOREIGN KEY (INSID) REFERENCES Insurance(INSID), FOREIGN KEY (CAID) REFERENCES Clinical_Activity(CAID))
- **Notation:** $\{E, T, C, I\}$
- **Functional Dependencies:**
 - $E \rightarrow T, I$
 - $C \rightarrow E$

Hospital Relation

- **Schema:** Hospital(HID: INTEGER, Name: STRING, City: STRING, Region: STRING, **PRIMARY KEY (HID)**)
- **Notation:** $\{H, N, Ci, R\}$
- **Functional Dependencies:**
 - $H \rightarrow N, Ci, R$
 - $Ci \rightarrow R$

Department Relation

- **Schema:** Department(Dep_ID: INTEGER, Name: STRING, Speciality: STRING, HID: INTEGER, **PRIMARY KEY (Dep_ID)**, FOREIGN KEY (HID) REFERENCES Hospital(HID))
- **Notation:** $\{D, N, S, H\}$
- **Functional Dependencies:**
 - $D \rightarrow N, S, H$

Clinical Activity Relation

- **Schema:** Clinical_Activity(CAID: INTEGER, IID: INTEGER, Staff_ID: INTEGER, Dep_ID: INTEGER, Date: STRING, Time: STRING, FOREIGN KEY (IID) REFERENCES Patient(IID), FOREIGN KEY (Staff_ID) REFERENCES Staff(Staff_ID), FOREIGN KEY (Dep_ID) REFERENCES Department(Dep_ID))
- **Notation:** $\{C, I, S, D, T\}$
- **Functional Dependencies:**
 - $C \rightarrow I, S, D, T$

Appointment Relation

- **Schema:** Appointment(AppointmentID: INTEGER, Reason: STRING, Status: STRING, **PRIMARY KEY (AppointmentID)**, FOREIGN KEY (AppointmentID) REFERENCES Clinical_Activity(CAID))
- **Notation:** $\{A, R, St\}$
- **Functional Dependencies:**
 - $A \rightarrow R, St$

Emergency Relation

- **Schema:** Emergency(EmergencyID: INTEGER, Outcome: STRING, Triage_Level: STRING, **PRIMARY KEY (EmergencyID)**, FOREIGN KEY (EmergencyID) REFERENCES Clinical_Activity(CAID))
- **Notation:** $\{E, O, T\}$
- **Functional Dependencies:**
 - $E \rightarrow O, T$

Practitioner Relation

- **Schema:** Practitioner(Staff_ID: INTEGER, License_Number: STRING, Specialty: STRING, **PRIMARY KEY (Staff_ID)**, FOREIGN KEY (Staff_ID) REFERENCES Staff(Staff_ID))
- **Notation:** $\{S, L, Sp\}$

- **Functional Dependencies:**

– $S \rightarrow L, Sp$

Technical Relation

- **Schema:** Technical(Staff_ID: INTEGER, Modality: STRING, Certification: STRING, **PRIMARY KEY (Staff_ID)**, FOREIGN KEY (Staff_ID) REFERENCES Staff(Staff_ID))

- **Notation:** $\{S, Mo, Ce\}$

- **Functional Dependencies:**

– $S \rightarrow Mo, Ce$

Medication Relation

- **Schema:** Medication(MID: INTEGER, Class: STRING, Name: STRING, Form: STRING, Strength: STRING, Manufacturer: STRING, Active_Ingredient: STRING, **PRIMARY KEY (MID)**)

- **Notation:** $\{M, Cl, N, F, St, Ma, A\}$

- **Functional Dependencies:**

– $M \rightarrow Cl, N, F, St, Ma, A$

Stock Relation

- **Schema:** Stock(HID, MID, Unit_Price: FLOAT, Stock_Stamp: STRING, Reorder_Level: INTEGER, **PRIMARY KEY (HID, MID)**, FOREIGN KEY (HID) REFERENCES Hospital(HID), FOREIGN KEY (MID) REFERENCES Medication(MID))

- **Notation:** $\{H, D, U, Ts, R\}$

- **Functional Dependencies:**

– $\{H, D\} \rightarrow U, Ts, R$

Prescription Relation

- **Schema:** Prescription(PID: INTEGER, Date_Issued: STRING, CAID: INTEGER, **PRIMARY KEY (PID)**, FOREIGN KEY (CAID) REFERENCES Clinical_Activity(CAID))

- **Notation:** $\{P, D, C\}$

- **Functional Dependencies:**

– $P \rightarrow D, C$

Include Relation

-
- **Schema:** Include(PID, MID, Dosage: STRING, Duration: STRING, **PRIMARY KEY (PID, MID)**, FOREIGN KEY (PID) REFERENCES Prescription(PID), FOREIGN KEY (MID) REFERENCES Medication(MID))
 - **Notation:** $\{P, M, Du, Do\}$
 - **Functional Dependencies:**
 - $\{P, M\} \rightarrow Du, Do$

5 Discussion

Challenges Faced

During this lab, we encountered several significant difficulties in translating complex healthcare queries into formal database operations. The most demanding aspect was handling aggregation requirements such as counts and averages. While we primarily used the standard Relational Algebra operators covered in class, we ultimately had to employ the gamma (γ) operator to properly express these aggregation operations.

We also faced obstacles in maintaining query efficiency while ensuring semantic correctness when translating between Relational Algebra and SQL. Another challenge was coordinating team understanding of complex query requirements to ensure consistent implementation across both formal and practical representations.

Observations

Throughout the implementation, we noticed that healthcare data relationships often required multi-step operations that combined both set operations and joins. The process revealed how small misunderstandings in relationship cardinalities could significantly impact query results.

We observed that creating intermediate relations improved both the readability and maintainability of complex queries. The translation between Relational Algebra and SQL also highlighted important differences in how these languages handle certain operations—particularly when dealing with the γ operator for aggregation functions.

Lessons Learned

This project significantly enhanced our understanding of Relational Algebra's practical application in real-world scenarios, particularly in handling complex healthcare data relationships. We developed a deeper appreciation for Functional Dependencies and their role in maintaining database integrity.

The process of translating Relational Algebra expressions to SQL improved our ability to think logically about query structure and optimization. Working with the γ operator provided valuable insights into handling aggregation within formal database frameworks, while the Functional Dependency analysis strengthened our understanding of database design principles.

6 Conclusion

This deliverable successfully demonstrated the application of advanced database concepts to the MNHS healthcare system. Through the formulation of complex queries in both Relational Algebra and SQL, coupled with comprehensive functional dependency analysis, we have validated the robustness of our database design. The project not only reinforced theoretical database principles but also provided practical experience in handling real-world healthcare data scenarios, preparing us for more advanced database management challenges in professional environments.